

School of Computer Science and Artificial Intelligence

Lab Assignment # 11.1

Program	: B. Tech (CSE)
Specialization	: CSE
Course Title	: AI Assisted coding
Course Code	: 23CS002PC304
Semester	: II
Academic Session	: 2025-2026
Name of Student	: A VamshiKrishna
Enrollment No.	: 2403A51L47
Batch No.	: 52
Date	: 24-02-2026

Task -1 : SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts

Prompt: Welcome to Smart Contact Manager!

Choose implementation:

1. Array-Based
2. Linked List-Based

Enter 1 or 2:

--- Menu ---

1. Add Contact
2. Search Contact
3. Delete Contact
4. Display All Contacts
5. Exit

Choose an option (1-5):

Enter name:

Enter phone number:

Enter name to search:

Found: Alice - 12345

Contact not found.

Enter name to delete:

Contact 'Alice' deleted.

Contact 'Alice' not found.

No contacts found.

Exiting Smart Contact Manager. Goodbye!

```
class ArrayContactManager:
    def __init__(self):
        self.contacts = []

    def add_contact(self, name, phone):
        self.contacts.append({"name": name, "phone": phone})
        print(f"Contact '{name}' added.")

    def search_contact(self, name):
        for contact in self.contacts:
            if contact["name"].lower() == name.lower():
                return contact
        return None

    def delete_contact(self, name):
        for i, contact in enumerate(self.contacts):
            if contact["name"].lower() == name.lower():
                self.contacts.pop(i)
                print(f"Contact '{name}' deleted.")
                return True
        print(f"Contact '{name}' not found.")
        return False

    def display_contacts(self):
        if not self.contacts:
            print("No contacts found.")
            return
        print("Array-based Contacts List:")
        for contact in self.contacts:
            print(f"{contact['name']}: {contact['phone']}")

class Node:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
        self.next = None

class LinkedListContactManager:
    def __init__(self):
        self.head = None

    def add_contact(self, name, phone):
        new_node = Node(name, phone)
        new_node.next = self.head
```

```

def add_contact(self, name, phone):
    self.head = new_node
    print(f"Contact '{name}' added.")

def search_contact(self, name):
    current = self.head
    while current:
        if current.name.lower() == name.lower():
            return {"name": current.name, "phone": current.phone}
        current = current.next
    return None

def delete_contact(self, name):
    current = self.head
    prev = None
    while current:
        if current.name.lower() == name.lower():
            if prev:
                prev.next = current.next
            else:
                self.head = current.next
            print(f"Contact '{name}' deleted.")
            return True
        prev = current
        current = current.next
    print(f"Contact '{name}' not found.")
    return False

def display_contacts(self):
    current = self.head
    if not current:
        print("No contacts found.")
        return
    print("Linked List-based Contacts List:")
    while current:
        print(f"{current.name}: {current.phone}")
        current = current.next

def demo():
    print("\n--- Array-Based Contact Manager Demo ---")
    array_manager = ArrayContactManager()
    array_manager.add_contact("Alice", "12345")
    array_manager.add_contact("Bob", "67890")
    array_manager.display_contacts()
    array_manager.display_contacts()
    print("Searching for Bob:", array_manager.search_contact("Bob"))
    array_manager.delete_contact("Alice")
    array_manager.display_contacts()

    print("\n--- Linked List-Based Contact Manager Demo ---")
    linked_manager = LinkedListContactManager()
    linked_manager.add_contact("Charlie", "54321")
    linked_manager.add_contact("Diana", "09876")
    linked_manager.display_contacts()
    print("Searching for Diana:", linked_manager.search_contact("Diana"))
    linked_manager.delete_contact("Charlie")
    linked_manager.display_contacts()

if __name__ == "__main__":
    demo()

```

```
Array-based Contacts List:
Bob: 67890

--- Linked List-Based Contact Manager Demo ---
Contact 'Charlie' added.
Contact 'Diana' added.
Linked List-based Contacts List:
Diana: 09876
Charlie: 54321
Searching for Diana: {'name': 'Diana', 'phone': '09876'}
Contact 'Charlie' deleted.
Linked List-based Contacts List:
Diana: 09876

C:\Users\atur\OneDrive\Desktop\Ai_assistanr>
```

Explanation :

- This code defines two contact manager classes: ArrayContactManager and LinkedListContactManager.
- Each class provides methods to add, search, delete, and display contacts. The demo function demonstrates the functionality of both implementations.

Task 2:

1. Implement a Queue (FIFO) to manage book requests.
2. Extend the system to a Priority Queue, prioritizing faculty requests.
3. Use GitHub Copilot to assist in generating:
 - enqueue() method
 - dequeue() method
4. Test the system with a mix of student and faculty requests.

prompt: Implement a queue data structure to manage book requests in a library. Each request should include the requester's name, type (student or faculty), and the book title. The queue should follow the First-In-First-Out (FIFO) principle. Include methods to enqueue a request, dequeue a request, and display the current queue.

```
class BookQueue:
    def __init__(self):
        self.queue = []

    # Enqueue a request (add to end)
    def enqueue(self, requester_type, name, book_title):
        request = {"type": requester_type, "name": name, "book": book_title}
        self.queue.append(request)
        print(f"Request added: {name} ({requester_type}) for '{book_title}'")

    # Dequeue a request (remove from front)
    def dequeue(self):
        if not self.queue:
            print("Queue is empty.")
            return None
        request = self.queue.pop(0)
        print(f"Processing request: {request['name']} ({request['type']}) for '{request['book']}'")
        return request

    # Display queue
    def display_queue(self):
        if not self.queue:
            print("Queue is empty.")
            return
        print("Current Queue:")
        for req in self.queue:
            print(f"{req['name']} ({req['type']}) - '{req['book']}'")

# -----
# Test FIFO Queue
# -----
print("=== FIFO Queue Test ===")
fifo_queue = BookQueue()
fifo_queue.enqueue("Student", "Alice", "Data Science 101")
fifo_queue.enqueue("Faculty", "Dr. Smith", "Advanced AI")
fifo_queue.enqueue("Student", "Bob", "Python Programming")

fifo_queue.display_queue()
fifo_queue.dequeue()
fifo_queue.dequeue()
fifo_queue.display_queue()
```

```
C:\Users\atur\OneDrive\Desktop\Ai_assistanr>C:/Python314/python.exe d
=== FIFO Queue Test ===
Request added: Alice (Student) for 'Data Science 101'
Request added: Dr. Smith (Faculty) for 'Advanced AI'
Request added: Bob (Student) for 'Python Programming'
Current Queue:
Alice (Student) - 'Data Science 101'
Dr. Smith (Faculty) - 'Advanced AI'
Bob (Student) - 'Python Programming'
Processing request: Alice (Student) for 'Data Science 101'
Processing request: Dr. Smith (Faculty) for 'Advanced AI'
Current Queue:
Bob (Student) - 'Python Programming'
```

Explanation :

- The FIFO (First In, First Out) queue ensures that requests are processed in the order they were added.
- In this example, Alice's request is processed first, followed by Dr. Smith's request.
- Bob's request remains in the queue and is processed last.

Task 3:

Emergency Help Desk (Stack Implementation)

Scenario

SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach.

Prompt:

1. Implement a Stack to manage support tickets.
2. Provide the following operations:
 - push(ticket)
 - pop()
 - peek()
3. Simulate at least five tickets being raised and resolved.
4. Use GitHub Copilot to suggest additional stack operations such as:
 - o Checking whether the stack is empty
 - o Checking whether the stack is full (if applicable)

```
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, ticket):
        self.stack.append(ticket)
        print(f'Ticket "{ticket}" has been added to the stack.')

    def pop(self):
        if not self.is_empty():
            ticket = self.stack.pop()
            print(f'Ticket "{ticket}" has been resolved and removed from the stack.')
            return ticket
        else:
            print('No tickets to resolve. The stack is empty.')
            return None

    def peek(self):
        if not self.is_empty():
            print(f'The next ticket to be resolved is: "{self.stack[-1]}"')
            return self.stack[-1]
        else:
            print('No tickets in the stack to peek at.')
            return None

    def is_empty(self):
        return len(self.stack) == 0

# Simulating ticket management
help_desk = Stack()
help_desk.push("Issue with Wi-Fi connectivity")
help_desk.push("Software installation request")
help_desk.push("Password reset")
help_desk.push("Printer not working")
help_desk.push("Email access issue")
help_desk.peek()
help_desk.pop()
help_desk.peek()
help_desk.pop()
help_desk.pop()
help_desk.pop()
help_desk.pop()
help_desk.pop() # Attempting to pop from an empty stack
```

```
C:\Users\atur\OneDrive\Desktop\Ai_assistanr>python -u "c:\Users\atur\OneDrive\Desktop\Ai_assistanr\p.py"
Ticket "Issue with Wi-Fi connectivity" has been added to the stack.
Ticket "Software installation request" has been added to the stack.
Ticket "Password reset" has been added to the stack.
Ticket "Printer not working" has been added to the stack.
Ticket "Email access issue" has been added to the stack.
The next ticket to be resolved is: "Email access issue"
Ticket "Email access issue" has been resolved and removed from the stack.
The next ticket to be resolved is: "Printer not working"
Ticket "Printer not working" has been resolved and removed from the stack.
Ticket "Password reset" has been resolved and removed from the stack.
Ticket "Software installation request" has been resolved and removed from the stack.
Ticket "Issue with Wi-Fi connectivity" has been resolved and removed from the stack.
No tickets to resolve. The stack is empty.

C:\Users\atur\OneDrive\Desktop\Ai_assistanr>
```

Explanation:

- This code defines a Stack class to manage help desk tickets using a Last In, First Out (LIFO) approach.
- The Stack class includes methods to push new tickets, pop resolved tickets, peek at the next ticket, and check if the stack is empty.

Task 4: Hash Table

Objective

To implement a Hash Table and understand collision handling.

Prompt: Implement a hash table in Python that supports insert, search, and delete operations. Use chaining to handle collisions. Provide well-commented code for each method.

```
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)] # Create a list of empty lists for chaining

    def hash_function(self, key):
        return hash(key) % self.size # Simple hash function using built-in hash

    def insert(self, key, value):
        index = self.hash_function(key)
        # Check if the key already exists and update it
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value) # Update existing key
                return
        # If key does not exist, add new key-value pair
        self.table[index].append((key, value))

    def search(self, key):
        index = self.hash_function(key)
        for k, v in self.table[index]:
            if k == key:
                return v # Return the value if key is found
        return None # Return None if key is not found

    def delete(self, key):
        index = self.hash_function(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i] # Remove the key-value pair
                return True # Return True if deletion is successful
        return False # Return False if key is not found

# Example usage
if __name__ == "__main__":
    ht = HashTable()
    ht.insert("name", "Alice")
    ht.insert("age", 30)
    print(ht.search("name")) # Output: Alice
    print(ht.search("age")) # Output: 30
    ht.delete("name")
    print(ht.search("name")) # Output: None
    print(ht.search("age")) # Output: 30
```



```
C:\Users\atur\OneDrive\Desktop\Ai_assistanr>C:/Python314/
Alice
30
None
30
```

Explanation:

- The HashTable class implements a hash table using chaining to handle collisions.
- The insert, search, and delete methods allow for adding, retrieving, and removing key-value pairs from the hash table.

Task 5:

Real-Time Application Challenge

Scenario

Design a Campus Resource Management System with the following features:

- Student Attendance Tracking
- Event Registration System
- Library Book Borrowing
- Bus Scheduling System
- Cafeteria Order Queue

Prompt :

1. Choose the most appropriate data structure for each feature.
2. Justify your choice in 2–3 sentences.
3. Implement one selected feature using AI-assisted code generation.

Expected Outcome

- Mapping table: Feature → Data Structure → Justification
- One fully working Python implementation

```
# Mapping table: Feature → Data Structure → Justification
mapping_table = {
    "Student Attendance Tracking": ("Hash Table", "A hash table allows for efficient storage and retrieval of student attendance records using unique student IDs as keys."),
    "Event Registration System": ("Queue", "A queue is suitable for managing event registrations as it follows a first-come, first-served approach, ensuring fairness in processing registrations."),
    "Library Book Borrowing": ("Linked List", "A linked list can efficiently manage the dynamic nature of book borrowing and returning, allowing for easy insertion and deletion of records."),
    "Bus Scheduling System": ("Graph", "A graph can represent the bus routes and stops effectively, allowing for complex scheduling and route optimization."),
    "Cafeteria Order Queue": ("Queue", "A queue is ideal for managing cafeteria orders as it ensures that orders are processed in the order they were received.")
}

# Implementing the Student Attendance Tracking feature using a Hash Table
class AttendanceTracker:
    def __init__(self):
        self.attendance_records = {}

    def mark_attendance(self, student_id, date):
        if student_id not in self.attendance_records:
            self.attendance_records[student_id] = []
        self.attendance_records[student_id].append(date)

    def get_attendance(self, student_id):
        return self.attendance_records.get(student_id, [])

# Example usage
attendance_tracker = AttendanceTracker()
attendance_tracker.mark_attendance("S001", "2024-09-01")
attendance_tracker.mark_attendance("S001", "2024-09-02")
attendance_tracker.mark_attendance("S002", "2024-09-01")
print(attendance_tracker.get_attendance("S001")) # Output: ['2024-09-01', '2024-09-02']
print(attendance_tracker.get_attendance("S002")) # Output: ['2024-09-01']
print(attendance_tracker.get_attendance("S003")) # Output: []
```

```
C:\Users\aelur\OneDrive\Desktop\Ai_assistanr>C:/Python314/python.exe  
['2024-09-01', '2024-09-02']  
['2024-09-01']  
[]
```

Explanation:

- The hash table (dictionary) is efficient for storing and retrieving attendance records.
- It allows $O(1)$ average time complexity for both marking and retrieving attendance data.